

GUÍA DE TESTING — NIVEL 2: ROLES

SEGURIDAD EN SUPABASE

Roles de PostgreSQL en Supabase

2.1 — Tests: Roles Predefinidos de Supabase

Estos tests verifican la **jerarquía de roles** (anon, authenticated, service_role) y cómo PostgREST asigna el rol según el JWT.

Test 2.1.1 — Identificar el rol activo desde SQL Editor

Verificar qué rol y funciones auth retorna el SQL Editor cuando se ejecuta directamente (como superusuario).

SQL Editor:

```
-- Ejecutar directamente en el SQL Editor de Supabase
SELECT
  current_user      AS usuario_pg,
  current_setting('role') AS rol_sesion,
  auth.uid()        AS auth_uid,
  auth.role()        AS auth_role,
  auth.email()       AS auth_email;
```

Acción / Test	Rol	Resultado esperado	☑/✗
Ejecutar query en SQL Editor	Superuser	usuario_pg = 'postgres', auth.uid() = NULL	☑
Verificar auth.role()	Superuser	NULL o vacío (no hay JWT en SQL Editor)	☑

Concepto clave:

El SQL Editor ejecuta como superusuario (postgres), que bypassa RLS.
 Por eso auth.uid() retorna NULL: no hay JWT en una sesión directa.
 Esto demuestra que el SQL Editor NO simula un usuario de la app.

Test 2.1.2 — Comparar roles: anon vs authenticated vs service_role

Realizar el mismo SELECT a **usuarios_perfil** con 3 tipos de credenciales distintas y comparar resultados.

Request A — Sin JWT (rol anon):

```
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=id,nombre_completo,rol

Headers:
  apikey: {{ANON_KEY}}
  -- SIN header Authorization
```

Request B — Con JWT de estudiante (rol authenticated):

```
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=id,nombre_completo,rol
```

Headers:

apikey: {{ANON_KEY}}

Authorization: Bearer {{JWT_ALUMNO}}

Request C — Con service_role key (bypasea RLS):

```
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=id,nombre_completo,rol
```

Headers:

apikey: {{SERVICE_ROLE_KEY}}

Authorization: Bearer {{SERVICE_ROLE_KEY}}

Acción / Test	Rol	Resultado esperado	☑/✕
Request A: solo anon key, sin JWT	anon	Array vacío [] (no hay política TO anon)	☑
Request B: JWT del estudiante	authenticated	1 fila: solo el perfil del estudiante	☑
Request C: service_role key	service_role	TODAS las filas (bypasea RLS)	☑

Regla crítica:

Este test demuestra visualmente la diferencia entre los 3 roles:

- anon: denegado por defecto (sin política específica)
- authenticated: filtrado por RLS (solo ve su perfil)
- service_role: acceso total, ignora todas las políticas

⚠ NUNCA usar service_role en el frontend. Si se filtra, game over.

Test 2.1.3 — service_role puede escribir sin restricciones

Verificar que el service_role puede hacer INSERT/UPDATE en cualquier tabla ignorando RLS y triggers de validación de negocio.

Postman/Insomnia:

```
-- service_role inserta un perfil con rol 'admin' directamente
POST {{SUPABASE_URL}}/rest/v1/usuarios_perfil

Headers:
  apikey: {{SERVICE_ROLE_KEY}}
  Authorization: Bearer {{SERVICE_ROLE_KEY}}
  Content-Type: application/json
  Prefer: return=representation

Body:
{
  "id": "00000000-0000-0000-0000-000000000099",
  "nombre_completo": "Test Service Role",
  "rol": "admin"
}
```

SQL Editor (verificación y limpieza):

```
-- Verificar que se insertó sin pasar por RLS
SELECT id, nombre_completo, rol FROM public.usuarios_perfil
WHERE id = '00000000-0000-0000-0000-000000000099';

-- Limpiar el dato de prueba
DELETE FROM public.usuarios_perfil
WHERE id = '00000000-0000-0000-0000-000000000099';
```

Acción / Test	Rol	Resultado esperado	☑/✗
service_role INSERT con ID arbitrario	service_role	INSERT exitoso, sin validación RLS	☑
Verificar que el registro existe	SQL Editor	Fila creada con rol = 'admin'	☑
Contrastar: authenticated con ID ajeno	authenticated	WITH CHECK falla (id != auth.uid())	✗

2.2 — Tests: Cláusula TO en Políticas

Estos tests verifican políticas dirigidas a **roles específicos** usando la cláusula TO (anon, service_role).

Preparación: Crear políticas específicas por rol

Ejecutá este SQL en el Editor para crear las políticas del módulo (si aún no las tenés):

```
-- Política para anon: solo lectura de materias activas
CREATE POLICY "materias_select_publico"
  ON public.materias
  FOR SELECT
  TO anon
  USING ( activa = true );

-- Política para service_role: update total en materias
CREATE POLICY "materias_update_service"
  ON public.materias
  FOR UPDATE
  TO service_role
  USING (true)
  WITH CHECK (true);
```

Test 2.2.1 — anon puede leer materias activas (catálogo público)

Política verificada: **materias_select_publico** (TO anon)

Postman/Insomnia:

```
GET {{SUPABASE_URL}}/rest/v1/materias?select=codigo,nombre,semestre

Headers:
  apikey: {{ANON_KEY}}
  -- SIN Authorization (simula visitante no logueado)
```

Acción / Test	Rol	Resultado esperado	☑/✕
anon lee materias	anon	Ve materias con activa = true	☑
Verificar que materias inactivas no aparecen	anon	TST-999 (inactiva) no visible	☑

Test 2.2.2 — anon NO puede leer otras tablas

Verificar que sin política TO anon, las demás tablas son inaccesibles.

Postman/Insomnia:

```
-- Intento 1: anon lee usuarios_perfil
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=*

Headers:
  apikey: {{ANON_KEY}}
```

```
-- Intento 2: anon lee calificaciones
GET {{SUPABASE_URL}}/rest/v1/calificaciones?select=*
```

Headers:
apikey: {{ANON_KEY}}

-- Intento 3: anon intenta INSERT en materias
POST {{SUPABASE_URL}}/rest/v1/materias

Headers:
apikey: {{ANON_KEY}}
Content-Type: application/json

Body:
{ "codigo": "HACK-001", "nombre": "Hackeada", "creditos": 1, "semestre": "2026-I" }

Acción / Test	Rol	Resultado esperado	☑/✗
anon lee usuarios_perfil	anon	Array vacío [] (sin política TO anon)	☑
anon lee calificaciones	anon	Array vacío []	☑
anon INSERT en materias	anon	Error 403: violates RLS policy	✗

Principio importante:

La política materias_select_publico solo permite SELECT TO anon.

No hay política INSERT TO anon, por lo que la escritura falla.

Cada operación (SELECT, INSERT, UPDATE, DELETE) requiere su propia política.

Test 2.2.3 — service_role puede actualizar cualquier materia

Política verificada: **materias_update_service** (TO service_role, USING/WITH CHECK true)

Postman/Insomnia:

```
-- service_role desactiva una materia (simula CRON job)
PATCH {{SUPABASE_URL}}/rest/v1/materias?codigo=eq.SIS-301

Headers:
  apikey: {{SERVICE_ROLE_KEY}}
  Authorization: Bearer {{SERVICE_ROLE_KEY}}
  Content-Type: application/json
  Prefer: return=representation

Body:
{ "activa": false }
```

Verificación y restauración:

```
-- Verificar que la materia fue desactivada
SELECT codigo, nombre, activa FROM public.materias WHERE codigo = 'SIS-301';

-- Restaurar para futuros tests
UPDATE public.materias SET activa = true WHERE codigo = 'SIS-301';
```

Acción / Test	Rol	Resultado esperado	☑/✕
service_role desactiva materia ajena	service_role	UPDATE exitoso, activa = false	☑
Contrastar: docente intenta el mismo UPDATE	Docente	Funciona SOLO si es SU materia	☑
Contrastar: estudiante intenta el UPDATE	Estudiante	0 filas afectadas (sin política UPDATE)	☑

Test 2.2.4 — authenticated NO usa políticas de service_role

Verificar que las políticas TO service_role son exclusivas y no aplican a usuarios autenticados normales.

SQL Editor (preparar escenario):

```
-- Crear materia SIN docente asignado (para que ningún docente la pueda editar por RLS)
INSERT INTO public.materias (codigo, nombre, creditos, docente_id, semestre)
VALUES ('ORF-001', 'Materia Sin Docente', 3, NULL, '2026-I');
```

Postman/Insomnia:

```
-- Docente intenta editar materia sin docente asignado
-- No tiene política propia (docente_id = NULL != auth.uid())
-- Y la política TO service_role NO le aplica
PATCH {{SUPABASE_URL}}/rest/v1/materias?codigo=eq.ORF-001

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_DOCENTE}}
  Content-Type: application/json
  Prefer: return=representation

Body:
{ "nombre": "Materia Robada" }
```

Limpieza:

```
DELETE FROM public.materias WHERE codigo = 'ORF-001';
```

Acción / Test	Rol	Resultado esperado	✓/✗
Docente edita materia sin dueño	Docente	0 filas afectadas (política service_role no aplica)	✓
Misma operación con service_role key	service_role	UPDATE exitoso (bypasea todo)	✓

2.3 — Tests: Privilegios a Nivel de Columna

Estos tests verifican **GRANT/REVOKE** a nivel de columna: una capa de defensa que controla qué **columnas** puede leer/escribir cada rol, independientemente de RLS.

Preparación: Aplicar privilegios de columna

Ejecutá este SQL en el Editor para configurar los privilegios del módulo:

```
-- 1. Revocar todos los permisos en la tabla
REVOKE ALL ON public.usuarios_perfil FROM authenticated;

-- 2. Conceder SELECT solo en columnas no sensibles
GRANT SELECT (id, nombre_completo, rol, carrera, avatar_url, activo)
  ON public.usuarios_perfil TO authenticated;

-- 3. El teléfono también visible (combinado con RLS, solo ve su fila)
GRANT SELECT (telefono)
  ON public.usuarios_perfil TO authenticated;

-- 4. Permitir UPDATE solo en campos editables
GRANT UPDATE (nombre_completo, carrera, telefono, avatar_url)
  ON public.usuarios_perfil TO authenticated;

-- 5. Permitir INSERT (para crear perfil al registrarse)
GRANT INSERT (id, nombre_completo, rol, carrera, telefono)
  ON public.usuarios_perfil TO authenticated;
```

Importante:

Después de REVOKE ALL, el rol authenticated solo puede acceder a las columnas explícitamente concedidas. Esto es independiente de RLS: aunque una política permita ver la fila, las columnas no concedidas dan error.

Test 2.3.1 — SELECT: columnas permitidas vs restringidas

Verificar que authenticated puede leer las columnas concedidas, pero **NO puede leer columnas no concedidas** como eliminado o created_at.

Test A — Leer columnas permitidas (debe funcionar):

```
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=id,nombre_completo,rol,carrera,telefono

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ALUMNO}}
```

Test B — Leer columna NO concedida (debe fallar):

```
-- Intentar leer 'eliminado' que NO fue concedida en el GRANT
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=id,nombre_completo,eliminado

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ALUMNO}}
```

Test C — Intentar SELECT * (todas las columnas):

```
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=*
```


Headers:
 apikey: {{ANON_KEY}}
 Authorization: Bearer {{JWT_ALUMNO}}

Acción / Test	Rol	Resultado esperado	☑/✗
SELECT columnas permitidas	Estudiante	Retorna datos correctamente	☑
SELECT columna 'eliminado' (no concedida)	Estudiante	Error 403: permission denied for column	✗
SELECT * (incluye columnas restringidas)	Estudiante	Error 403: permission denied	✗

Test 2.3.2 — UPDATE: campos editables vs protegidos

Verificar que authenticated puede actualizar nombre_completo, carrera, telefono y avatar_url, pero **NO puede actualizar rol, eliminado ni activo** directamente por privilegio de columna.

Test A — UPDATE campo permitido (debe funcionar):

```
PATCH {{SUPABASE_URL}}/rest/v1/usuarios_perfil?id=eq.{{UUID_ALUMNO}}
```

Headers:
 apikey: {{ANON_KEY}}
 Authorization: Bearer {{JWT_ALUMNO}}
 Content-Type: application/json
 Prefer: return=representation

Body:
 { "telefono": "+591 71234567" }

Test B — UPDATE campo NO concedido (debe fallar):

```
-- Intentar cambiar 'rol' directamente
-- Ahora falla por DOBLE protección: privilegio de columna + trigger
PATCH {{SUPABASE_URL}}/rest/v1/usuarios_perfil?id=eq.{{UUID_ALUMNO}}
```

Headers:
 apikey: {{ANON_KEY}}
 Authorization: Bearer {{JWT_ALUMNO}}
 Content-Type: application/json
 Prefer: return=representation

Body:
 { "rol": "admin" }

Acción / Test	Rol	Resultado esperado	☑/✗
UPDATE telefono (campo permitido)	Estudiante	UPDATE exitoso, teléfono cambia	☑
UPDATE rol (campo NO concedido)	Estudiante	Error 403: permission denied for column	✗
UPDATE activo (campo NO concedido)	Estudiante	Error 403: permission denied for column	✗
UPDATE eliminado (campo NO concedido)	Estudiante	Error 403: permission denied for column	✗

Defensa en profundidad — 3 capas en un solo campo:

Ahora el campo 'rol' tiene TRIPLE protección:

1. Privilegio de columna: REVOKE impide el UPDATE
2. Trigger: proteger_campos_sensibles fuerza OLD.rol
3. RLS: la política solo permite editar la propia fila

Esto es defensa en profundidad real: si una capa falla, las otras protegen.

Test 2.3.3 — INSERT: solo columnas permitidas

Verificar que al crear un perfil, solo se pueden incluir las columnas concedidas en el GRANT INSERT.

SQL Editor (preparar usuario temporal en auth):

```
-- NOTA: Para este test necesitás un usuario en auth.users cuyo perfil
-- aún no exista en usuarios_perfil. Podés crear uno nuevo vía signup
-- o borrar temporalmente el perfil de prueba.

-- Si querés testear directamente desde SQL Editor:
-- Verificar qué columnas tiene concedidas el rol authenticated
SELECT
  grantee, table_name, column_name, privilege_type
FROM information_schema.column_privileges
WHERE table_name = 'usuarios_perfil'
  AND grantee = 'authenticated'
ORDER BY privilege_type, column_name;
```

Postman/Insomnia — INSERT con columna no permitida:

```
-- Intentar insertar incluyendo 'activo' (no concedida en GRANT INSERT)
POST {{SUPABASE_URL}}/rest/v1/usuarios_perfil

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_NUEVO_USUARIO}}
  Content-Type: application/json
  Prefer: return=representation

Body:
{
  "id": "{{UUID_NUEVO_USUARIO}}",
  "nombre_completo": "Test Privilegios",
  "rol": "estudiante",
  "activo": false
}
```

Acción / Test	Rol		Resultado esperado	✓/ ✗
INSERT con columnas permitidas (id, nombre, rol)	authenticated		INSERT exitoso	✓
INSERT incluyendo 'activo' (no concedida)	authenticated		Error 403: permission denied for column	✗
INSERT incluyendo 'eliminado' (no concedida)	authenticated		Error 403: permission denied for column	✗

Test 2.3.4 — Verificar privilegios con information_schema

Consultar el catálogo de PostgreSQL para documentar exactamente qué privilegios tiene cada rol por columna.

SQL Editor:

```
-- Reporte completo de privilegios por columna
SELECT
  column_name AS columna,
  privilege_type AS privilegio,
  is_grantable AS transferible
FROM information_schema.column_privileges
```

```
WHERE table_schema = 'public'
  AND table_name = 'usuarios_perfil'
  AND grantee = 'authenticated'
ORDER BY column_name, privilege_type;
```

Resultado esperado:

Columna	SELECT	INSERT	UPDATE
id	✓	✓	✗
nombre_completo	✓	✓	✓
rol	✓	✓	✗
carrera	✓	✓	✓
telefono	✓	✓	✓
avatar_url	✓	✗	✓
activo	✓	✗	✗
eliminado	✗	✗	✗
created_at	✗	✗	✗
updated_at	✗	✗	✗

Checklist de Verificación — Nivel 2 Completo

Marcá cada test a medida que lo completés:

☒	Test	Descripción	OK
☐	2.1.1	Identificar rol activo desde SQL Editor (postgres/NULL)	
☐	2.1.2	Comparar respuestas: anon vs authenticated vs service_role	
☐	2.1.3	service_role escribe sin restricciones RLS	
☐	2.2.1	anon lee materias activas (catálogo público)	
☐	2.2.2	anon NO puede leer otras tablas ni escribir	
☐	2.2.3	service_role actualiza cualquier materia	
☐	2.2.4	authenticated NO usa políticas de service_role	
☐	2.3.1	SELECT: columnas permitidas OK, restringidas error	
☐	2.3.2	UPDATE: campos editables OK, protegidos error	
☐	2.3.3	INSERT: solo columnas concedidas en GRANT	
☐	2.3.4	Documentar privilegios con information_schema	